Factory Method Exercises

https://github.com/akarshiev

Purpose: Complete these exercises independently, add automated tests, and document how to run each solution. Starter code is intentionally omitted so the repository reflects your own implementation.

Exercise 1: Shape Factory

Build a factory that creates Circle, Rectangle, and Square objects. Each shape calculates area and perimeter.

Requirements

- Define a shared abstract Shape with area() and perimeter().
- Implement Circle, Rectangle, and Square.
- Implement ShapeFactory.create_shape(shape_type, *args).
- Support circle, rectangle, and square.
- Raise ValueError for unsupported shape types.
- Validate dimensions and argument counts.
- Provide describe() output with shape name, area, and perimeter.

Test and verify

- Create a circle with radius 5, rectangle with width 4 and height 6, and square with side 3.
- Check calculations and returned classes.
- Test unknown types, invalid dimensions, and missing arguments.

Exercise 2: Parameterized Document Factory

Create Text, PDF, and Spreadsheet document types with shared metadata and type-specific settings.

Requirements

- Define DocumentType enum values TEXT, PDF, and SPREADSHEET.
- Define an abstract Document with title, author, creation time, modification time, and content.
- Require open(), save(content), and get_metadata() methods.
- TextDocument supports encoding and line count.
- PDFDocument supports page size, security level, and page count.
- SpreadsheetDocument supports row/column dimensions and sheet count.
- Implement DocumentFactory.create_document(doc_type, title, author, **kwargs).
- Pass type-specific options to the correct constructor and reject invalid types or options clearly.

Test and verify

- Create a UTF-16 text document.
- Create a PDF with Legal page size and security level 2.
- Create a spreadsheet with 1,000 rows and 50 columns.
- Save sample content and verify metadata and modification time update.

Exercise 3: Extensible Notification Factory

Create a registry-based factory that supports registering notification types at runtime.

Requirements

- Define NotificationConfig with priority, retry_count, and timeout.
- Define an abstract Notification with recipient, message, config, status, send(), format_message(), and get_status().
- Implement EmailNotification, SMSNotification, and PushNotification. Email requires a subject; push requires a device token and may accept a badge count.
- Implement register_notification(name, notification_class), create_notification(notification_type, recipient, message, **kwargs), and list_available_types().
- Validate registrations, duplicate names, and configuration.
- Handle unknown notification types consistently. Choose either None or a custom exception and document the decision.
- Sending may be simulated; no real external notification service is required.

Test and verify

- Register email, sms, and push types.
- Create and send each type, then inspect formatted messages and status.
- Test unknown types, duplicate registration, and missing required arguments.
- Register a new notification class without changing the factory implementation.

Repository checklist

- Separate shared abstractions, concrete products, and factory logic into understandable modules.
- Include a README with goals, design decisions, setup instructions, and test commands.
- Use automated tests for object selection, calculations, metadata, registration, and invalid input.
- Use type hints and docstrings where useful.
- Do not commit virtual environments, caches, generated files, or secrets.
- Explain how the Factory pattern separates object creation from client code and supports extension.